CAS CS 538. Solutions to Problem Set 7

Due electronically via Gradescope, Monday March 30, 2026 11:59pm

Problem 1. (25 points) Consider the following MAC (a variant of this was used for Wi-Fi encryption in 802.11b WEP). Let F be a PRF defined over $(\mathcal{K}, \mathcal{R}, \mathcal{X})$ where $\mathcal{X} := \{0,1\}^{32}$. Let CRC32 be any efficiently computable function mapping $m \in \{0,1\}^{\leqslant \ell}$ to a 32-bit string (in 802.11b, it was a simple and popular error-detecting code meant to detect random errors). Define the following MAC system (S, V):

$$S(k,m) := \left\{ r \xleftarrow{R} \mathcal{R}, t \twoheadleftarrow F(k,r) \oplus \text{CRC32}(m), \text{output } (r,t) \right\},$$
$$V(k,m,(r,t)) := \{\text{accept if } t = F(k,r) \oplus \text{CRC32}(m) \text{ and reject otherwise}\}.$$

Show that this MAC system is insecure.

Solution. The main issue with this MAC setup is the separation of the nonce r from the msg m via XOR. Basically, the MAC reveals enough info about the hidden value $t = F(k,r)$ to allow an adv. to compute a new valid MAC for a different msg using the same nonce.

Adversary $\mathcal{A}$

Let $\mathcal{A}$ be an efficent adv attacking the defined MAC protocol:

- $\mathcal{A}$ chooses some rnd msg $m_1 \in \mathcal{M}$.
- $\mathcal{A}$ queries the signing oracle $S(k, m_1)$, and gets (r_1, t_1).
- $\mathcal{A}$ then chooses a rnd msg $m_2 \in \mathcal{M}$ such that $m_2 \neq m_1$.
- $\mathcal{A}$ then computes t_2:
$$t_2 = t_1 \oplus \text{CRC32}(m_1) \oplus \text{CRC32}(m_2)$$
- $\mathcal{A}$ outputs the candidate forgery $(m_2, (r_1, t_2))$.

Analysis

Note that, in order for $\mathcal{A}$ to win, it must achieve the following properties:

1. The msg, m_2, was never queried to the signing oracle.
2. The verification algorithm accepts the pair: $V(k, m_2, (r_1, t_2)) = \text{ACCEPT}$.

Freshness By construction, $\mathcal{A}$ only queried m_1. Since $m_2 \neq m_1$, freshness is guaranteed.

Validity From the signing query, $\mathcal{A}$ knows that $t_1 = F(k, r_1) \oplus \text{CRC32}(m_1)$. It is now possible to rearrange this, such that we can isolate the PRF output:

$$F(k, r_1) = t_1 \oplus \text{CRC32}(m_1)$$

Now, consider the verification equation for the forgery $(m_2, (r_1, t_2))$. The verifier checks if:

$$t_2 \stackrel{?}{=} F(k, r_1) \oplus \text{CRC32}(m_2)$$

By subbing the value of t_2, computed by $\mathcal{A}$:

$$\begin{aligned} t_2 &= (t_1 \oplus \text{CRC32}(m_1)) \oplus \text{CRC32}(m_2) \\ &= F(k, r_1) \oplus \text{CRC32}(m_2) \end{aligned}$$

The equation holds perfectly. Therefore, V will always output ACCEPT.

End

Since $\mathcal{A}$ wins the game with probability 1:

$$\text{MACadv}[\mathcal{A}, \text{MAC}] = 1$$

Since 1 is not negligible, this proves that the MAC protocol is insecure. □

Problem 2. (30 points) We want to build a MAC system $\mathcal{I}$ using two MAC system $\mathcal{I}_1 = (S_1, V_1)$ and $\mathcal{I}_2 = (S_2, V_2)$, so that if at some time one of $\mathcal{I}_1$ or $\mathcal{I}_2$ is broken (but not both) then $\mathcal{I}$ is still secure. Put another way, we want to construct $\mathcal{I}$ from $\mathcal{I}_1$ and $\mathcal{I}_2$ such that $\mathcal{I}$ is secure if either $\mathcal{I}_1$ or $\mathcal{I}_2$ is secure. This is also called a *MAC combiner*.

For this problem, suppose that $\mathcal{I}_1$ and $\mathcal{I}_2$ are deterministic MAC systems (see the definition on textbook page 217 in Section 6.1), and that both have tag space $\{0,1\}^n$. Define the deterministic MAC system $\mathcal{I} = (S, V)$, where

$$S((k_1, k_2), m) := S_1(k_1, m) \oplus S_2(k_2, m).$$

Show that $\mathcal{I}$ is a MAC combiner of $\mathcal{I}_1$ and $\mathcal{I}_2$.

Solution. This can be proved through a reduction. Firstly, note the construction of $\mathcal{I}$:

$$S_1(k_1, m) \oplus S_2(k_2, m) = \mathcal{I}_1 \oplus \mathcal{I}_2$$

By logic and the laws of XOR, we are essentially just proving $(\mathcal{I}_1 \to \mathcal{I}) \vee (\mathcal{I}_2 \to \mathcal{I})$. So by proving that $\mathcal{I}_1$ is secure, we prove $\mathcal{I}$ is secure. The proof for $\mathcal{I}_2$ will be the same thing.

Game

Let $\mathcal{A}$ be the efficent adversary attacking $\mathcal{I}$, assumed to have some adv ϵ. Then, let $\mathcal{B}$ be another efficent adversary, attacking $\mathcal{I}_1$. Assume the contrapositive: If $\mathcal{I}$ is insecure, then $\mathcal{I}_1$ is insecure.

- $\mathcal{B}$ can query $S_1(k_1, \cdot)$. $\mathcal{A}$'s query will be intercepted by $\mathcal{B}$.
- Before anything, $\mathcal{B}$ will choose rnd $k_2 \to \mathcal{K}_2$.
  - $\mathcal{B}$ is now essentially simulating $\mathcal{I}_2$. This is possible, as we assume that we know how $\mathcal{I}_2$ works.
- Whenever $\mathcal{A}$ makes a query with m:
  - $\mathcal{B}$ forwards query to Challenger, gets $t_1 = S_1(k_1, m)$.
  - $\mathcal{B}$ then computes $t_2 = S_2(k_2, m)$.
  - $\mathcal{B}$ computes $t = t_1 \oplus t_2$.
  - $\mathcal{B}$ sends t to $\mathcal{A}$
- $\mathcal{A}$ will output some forgery (m^*, t^*). $\mathcal{B}$ intercepts this.
- $\mathcal{B}$ then computes:
  - $t_2^* = S_2(k_2, m^*)$
  - $t_1^* = t^* \oplus t_2^*$
- $\mathcal{B}$ returns forgery (m^*, t_1^*).

Analysis

Validity If $\mathcal{A}$ wins, by initial assumption, then (m^*, t^*) is a valid forged tag. By the definition of $\mathcal{I}$:

$$t^* = S_1(k_1, m^*) \oplus S_2(k_2, m^*)$$

Therefore what $\mathcal{B}$ computes is then:

$$\begin{aligned} t_1^* &= t^* \oplus S_2(k_2, m^*) \\ &= (S_1(k_1, m^*) \oplus S_2(k_2, m^*)) \oplus S_2(k_2, m^*) \\ &= S_1(k_1, m^*) \end{aligned}$$

By the initial assumption, this means that $\mathcal{B}$'s forgery will also then be valid.

Freshness If $\mathcal{A}$ wins, m^* is never sent to $\mathcal{I}$. $\mathcal{B}$ always forwards $\mathcal{A}$'s queries to $\mathcal{I}_1$. Therefore, if $\mathcal{A}$ has won, then $\mathcal{B}$ hasn't sent any new message m^* to $\mathcal{I}_1$. Therefore $\mathcal{B}$'s forgery is a valid existential tag for $\mathcal{I}_1$.

End

Since, when $\mathcal{A}$ wins, $\mathcal{B}$ also wins, the advantage of $\mathcal{B}$ will be at least that of $\mathcal{A}$:

$$\mathsf{MACadv}[\mathcal{B}, \mathcal{I}_1] \geqslant \mathsf{MACadv}[\mathcal{A}, \mathcal{I}]$$

By our initial assumption, then $\mathcal{B}$ will therefore then be non-negligible. This proves the contrapositive, therefore proving the OG statement. $\square$

Problem 3. (45 points) For this problem, it may be helpful to review discussion 7 and section 9.4.2 in the textbook.

Consider the following candidate authenticated encryption scheme. The idea is to apply a MAC to a plaintext, add a checksum value to detect decryption errors[1], and then encrypt the plaintext, MAC, and checksum value together.

Let $\mathcal{E} = (E, D)$ be a block cipher with $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0,1\}^{128}$, and $\mathcal{I} = (S, V)$ be a MAC with tag space $\{0,1\}^{128}$. For the checksum, we'll use a function $\mathsf{Checksum} : \{0,1\}^* \to \{0,1\}^8$ that outputs a one-byte tag.

Define the scheme $\mathcal{E}' = (E', D')$ as follows:

$E'((k_e, k_m), m)$:
$t \twoheadleftarrow S(k_m, m)$ $x \twoheadleftarrow \mathsf{Checksum}(m \| t)$ $c \twoheadleftarrow$ encryption of $m \| t \| x \| 0^{120}$ under k_e, using E in randomized CBC mode Return c

$D'((k_e, k_m), c)$
$(m \| t \| x \| 0^{120}) \twoheadleftarrow$ decryption of c under k_e, using D in randomized CBC mode If $x \neq \mathsf{Checksum}(m \| t)$: Output reject Elif $V(k_m, m, t) = \mathsf{reject}$: Output reject Else: Output m

Notice that E' pads the plaintext with 120 bits of zeros. You may assume that all message sizes are multiples of 128 bits. This ensures that the second-to-last block of every ciphertext is an encryption of the tag, and the last block is an encryption of the checksum followed by 120 zeros.

Prove that $\mathcal{E}'$ is not AE-secure.

Solution. This protocol's main issue is with WHERE the checksum check is placed for the decryption. At decryption, checksum is verified **before** the MAC, allowing an adversary to learn information about the plaintext before the MAC.

Idea

Note two key properties regarding this question:

- **Checksum Before MAC:** D' checks $x = \mathsf{Checksum}(m \| t)$ before verifying $V(k_m, m, t)$. This means that a REJECT at the checksum stage reveals different information than a REJECT at the MAC stage.
- **CBC Malleability:** Then, by the malleability of CBC blocks, it is possible to alter the ciphertext block C_{i-1} such that it XORs the following modification into plaintext block P_i:

$$P_i' = D(k_e, C_i) \oplus (C_{i-1} \oplus \Delta) = P_i \oplus \Delta$$

Game

Let $\mathcal{A}$ be an efficent adversary atking $\mathcal{E}'$:

[1] Of course, verifying the MAC also suffices to detect decryption errors. Adding the checksum is an attempt at redundancy.

- When $\mathcal{A}$ requests an encryption of message m, it gets $c = (C_0, C_1, \ldots, C_{n-1}, C_n)$. It will then do the following:
  - C_{n-1} encrypts the MAC tag t.
  - C_n encrypts the checksum $x \| 0^{120}$.
- $\mathcal{A}$ will then make $c' = (C_0, C_1, \ldots, C_{n-1}, C_{n-1})$ (just replace the last block with the second-to-last block).
- Now, $\mathcal{A}$ will send c' to the decryption oracle.
- Now, what $\mathcal{A}$ will do will depend on the timing of the decryption function:
  - If the code sends REJECT at **checksum check** stage, $\mathcal{A}$ will output some guess about plaintext.
  - If a REJECT is sent at the **MAC check** stage (or accepts), $\mathcal{A}$ will then output a different guess.
- $\mathcal{A}$ should run this entire game (with same message m) multiple times, in order to get fresh encs, with different IVs, to increase confidence.

Analysis

c' When c' is decrypted, the last plaintext block becomes:

$$P'_n = D(k_e, C_{n-1}) \oplus C_{n-2} = t \oplus C_{n-2}$$

CHECKSUM only passes iff the first byte of $P'_n == \mathsf{Checksum}(m \| t)$. By the definition of $\mathcal{E}'$, this should only occur w/ probability $1/256$ (per query).

Leak If the oracle accepts (or reaches MAC verification), $\mathcal{A}$ then learns that:

$$\text{First byte of } (t \oplus C_{n-2}) = \mathsf{Checksum}(m \| t)$$

Since $t = S(k_m, m)$ depends on m, this leaks info about the plaintext. By def of $\mathcal{E}'$, by around 256 queries, $\mathcal{A}$ should be able to recover one byte of m with high probability:

$$\Pr[\text{success}] = 1 - \left(\frac{255}{256}\right)^{256} \approx 0.63$$

CCA-insecure By repeating this process for different byte positions (9.4.2 describes this as just doing msg shifting iteratively), $\mathcal{A}$ should then be able to recover the entire plaintext m eventually. This breaks IND-CCA security, implying that AE-security is also broken.

End

Since $\mathcal{A}$ can recover plaintext info with non-negligible adv using polynomial queries:

$$\mathsf{AEadv}[\mathcal{A}, \mathcal{E}'] \geqslant \frac{1}{256}$$

This probability is **not negligible** (it is a constant). Therefore, $\mathcal{E}'$ is not AE-secure. $\square$